5WCM2008,

Data Science Development Exercise (COM)

Team Project

Group name: G-10

Team leader: Saffa Amar Kiani

Members:

Esha Shahid, Ayesha Shafiq, Abdullah Muhammad Rahmani, Aiza Saleem, Humera Saeed

Contents

Introduction:

This report describes the design, development, and implementation of a machine learning-enabled web application built using Python Flask and SQLite. The project was developed as part of a group project that utilized full-stack development and artificial intelligence techniques to address a real-world problem. Our application addresses the challenge of automating chest X-ray diagnoses by offering a digital solution to assist patients, clinicians, radiologists, and administrators in managing and interpreting medical imaging data.

The project integrates a trained machine-learning model that can classify chest X-ray pictures into diagnostic categories. Users interact with the system via a secure web interface that includes role-specific capabilities unique to each stakeholder. Patients can upload images and view AI-generated results, doctors can provide professional insights, radiologists can confirm or modify diagnoses, and administrators can manage user roles and access permissions.

In addition to developing the application, we used an agile development approach that included writing comprehensive user stories, creating low-fidelity prototypes, implementing role-based access control, and rigorous testing. This report documents our design process, essential functions, machine learning methodology, and collaborative development practices, such as the usage of GitHub for version control. The finished application highlights the potential of AI-powered solutions to improve healthcare support systems.

Team members & Contribution:

Group leader:

Saffa Amar Kiani

- **Designed:** Designed the lo-fi prototype in collaboration with another team member humera.
- **User stories:** Wrote one user story focused on the doctor's role in the application.
- **Test case developing and testing:** Developed and wrote one test case specifically for the health worker's functionalities and conducted testing.
- **Project management, progress tracking:** Led project management activities, including task distribution, progress tracking.
- **Team meetings:** coordinating team meetings with collaboration with team members & ensuring attendance in all meetings.
- **Writing report & documentation:** Contributed significantly to the final report, project planning, and documentation.
- **Quality checking & guidance:** Provided ongoing guidance and support to team members, ensuring high-quality outputs.
- **Motivation & encouragement**: Played an instrumental role in motivating and ensuring team cohesion throughout the project.

Members:

Esha Shahid

As Developer A, I focused on implementing health worker and patient features in our Flask-based web application. Below are my contributions:

1. User Stories Implemented

- Radiologist Examines X-ray Reports – Developed X-ray upload & auto-diagnosis.
- Patient Accesses X-ray Report – Enabled patients to securely view and download reports.

2. Frontend Development

- Designed and built HTML templates:
    - health_worker_dashboard.html
    - new_patient.html
    - patient_dashboard.html
- Applied CSS styles using:
    - health_worker.css, new_patient.css, patient_dashboard.css
- Focused on layout, form design, button consistency, and responsiveness.

3. Backend Development

- Implemented Flask routes:
    - /dashboard/health_worker, /new_patient, /dashboard/patient, and logout ()
- Integrated AI-based auto-diagnosis using the best ML model.
- Built new patient form with X-ray upload and live predictions.
- Filtered dashboard view based on the logged-in health worker.

4. Collaboration & Tools

- Used PyCharm for development and GitHub for version control.
- I handled GitHub commits, pushes, and pull requests.
- Abdullah used Ngrok to provide a live link for testing the app remotely.
- Worked jointly to ensure smooth integration in the GUI-based app.

5. Additional Contributions

- Performed manual testing and bug fixing to ensure stability.
- Learned how to link a website to the app during the process.
- Wanted to add animations but was limited by the free CSS tools in PyCharm.

6. Joint Contributions with Developer B

- Flask core setup (app.py)
- Login & register functionality
- SQLite DB connection & session management
- Directory structure & file organization
- Testing support (manual/unit)

Humera Saeed

- **DESIGN WORK**: Co-designed the Lo-Fi prototype using Draw.io, and created key pages like the login, Diagnosis and recommendation, add new patient, and patient details page.
- **USER STORIES:** Helped and explained the user stories and format to teammates. Suggested what each user needs do to in the system.
- **NON-FUNCTIONAL REQUIREMENTS:** Wrote nonfunctional requirements for the web app focusing on performance expectations and security needs.
- **TESTING:** Wrote and executed test case of doctor's recommendation for the app that included functionality and security tests.
- **TEAMWORK:** Worked closely with my teammate Saffa to divide the design work, we evaluated each other's work, gave feedback, and adjusted our designs in response to recommendations.
- **TEAMS MEETINGS:** Attended regular meetings, provided team support and ideas, and when needed assisted in clarifying task.

Abdullah Muhammad Rahmani

1. **Backend Implementation using Flask:**
   Developed core backend functionalities, especially for the Doctor dashboard, including review handling, status updates, recommendations, and dynamic patient data routing.

2. **Machine Learning Model Integration:**
   Successfully integrated the pneumonia detection model into the Flask app, enabled confidence score rendering, and connected it with patient record updates.

3. **Frontend Contribution (Doctor Side):**
   Created responsive CSS for dashboard pages and tables, styled recommendation sections, and implemented status badges for diagnosis clarity.

4. **Flask App Infrastructure:**
   Helped set up app.py, structured routes and templates, configured the SQLite database connection, and handled user session management and authentication.

5. **Testing and Debugging Support:**
   Participated in manual testing and reviewed code for errors, helped with debugging user login and dashboard redirections, and contributed to early unit test planning.

6. **Team Communication and Leadership:**
   Initiated and managed the team WhatsApp group via Canvas. Scheduled team meetings, provided guidance when team members were stuck, and ensured everyone stayed updated and aligned.

7. **GitHub Version Control:**
   Made regular commits, handled pull requests, resolved merge issues, and ensured a clean and consistent repository was maintained throughout the project.

Aiza Saleem

- Designed two user stories:
- Patient: View reports, prescriptions, and appointments.
- Radiologist: Upload X-rays, use AI for analysis, and add diagnostic notes.
- Included clear acceptance criteria for each story.
- Developed two test cases:
- Login functionality: Verified valid credentials and role-based redirection.
- Patient diagnosis page: Ensured proper viewing of patient records.
- Performed manual testing:
- Tested login credentials for correct authentication and dashboard access.
- Attended regular team meetings and actively participated in discussions.
- Provided guidance and problem-solving support to team members.
- Contributed to the usability, reliability, and healthcare compliance of the web application.

Ayesha Shafiq

- Contributed significantly to the planning, and testing phases of the project.
- Authored two of four key user stories:
- Patient diagnosis access
- Admin role management
- Helped guide the development process based on these stories.

- Wrote three of five non-functional requirements in the report:
- Usability
- Accessibility
- Availability
- Ensured the app addressed real-world constraints and user diversity.
- Created and demonstrated a core manual test case:
- Patient views report — validated this functionality in the system.
- Helped structure and review the final report, ensuring clarity and quality.
- Contributed constructive feedback during team discussions.
- Actively participated in the group video, presenting individual contributions

Low-Fidelity Prototype Part 1:

For the chest ray project, our Lo-Fi prototype presents the design for a pneumonia diagnosis web application created for Chest Ray, a non-for-profit organization that supports healthcare in low-resource environments. The main goal of the app is to support non-expert health workers to diagnosis pneumonia with AI, and allow the clinician to review the results remotely.
On the login page, users select their role to access the relevant dashboard. The doctors are directed to the patients record page which displays patient details like name, age, symptoms, diagnosis, confidence score, status (Pending/Received), and action buttons. Clicking a patient's name leads to the Diagnosis Details Page, which shows full patient information, uploaded chest X-ray and submit doctors recommendation.
Health workers access a similar patient records page, but with additional option to add a new patient. The Add patient page allows input of patient details like name, age, symptoms, contact and upload the chest X-ray. When submitted, the Diagnosis Result page shows the AI-generated diagnosis and confidence score, along with a "send to clinician" button for a review.
Patients can login and view a simplified Patient summary page, showing their details, diagnosis, symptoms, status, and the doctor's recommendation, with a logout option.

User Stories Part 2:

The user stories below reflect essential functionality in our web application as seen by diverse stakeholders. Each story follows a consistent format to provide clarity and consistency with agile development principles.

User Story 1: Radiologist Reviews Chest X-ray Reports

Narrative:
As a radiologist,
I want to upload and analyze chest X-ray images using AI,
So that I can quickly detect potential abnormalities and provide accurate diagnoses.

Acceptance Criteria:

Scenario 1: AI-Assisted Image Analysis
Given a radiologist is logged into the web app,
And they have a chest X-ray image ready for review,
When they upload the image for analysis,
Then the AI should process the image and highlight potential abnormalities,
And display the results with confidence scores and explanations.

User Story 2: Patient Accesses Their Diagnosis Report

Narrative:
As a patient,
I want to view my X-ray results online,
So that I can understand my diagnosis and seek further consultation if needed.

Acceptance Criteria:

Scenario 1: Viewing Diagnosis Results
Given a patient has undergone an X-ray scan,
And the radiologist has uploaded the diagnosis report,
When the patient logs into the web app,
Then they should be able to securely access their X-ray images and diagnosis report,
And download or share the report with their doctor if needed.

User Story 3: Doctor Reviews Patient's X-ray and Provides Recommendations

Narrative:
As a doctor,
I want to access my patients' X-ray results and provide recommendations,
So that I can offer appropriate treatment based on AI-assisted analysis.

Acceptance Criteria:

Scenario 1: Doctor Reviews and Adds Notes
Given a doctor is logged into the web app,
And they have a list of their patients' X-ray results,
When they select a specific patient's report,
Then they should see the AI-generated analysis along with image annotations,
And add their own observations or treatment recommendations

User Story 4: Developers Manage User Accounts and Permissions

Narrative:
As Developer A or Developer B,
I want to manage user accounts and assign roles via the backend,
So that only authorized users can access specific features of the web app.
Acceptance Criteria:

Scenario 1: Assigning and Updating User Roles
Given Developer A or B is managing the user database,
And they want to assign or update a user's role (e.g., doctor, patient, or radiologist),
When they access the SQLite database and modify the user's role entry,
Then the system should recognize the role and restrict or grant access accordingly,
And reflect the changes immediately across the app without a separate admin interface.

Non-functionality part 3:

The non-functional requirements listed below define the web application's key attributes in addition to its basic functionality. These help to ensure that the system is dependable, user-friendly, secure, and available to all authorized users.

USABILITY

The user interface demands simplicity through big buttons and clear font alongside descriptive labels. The text needs high color contrast to enable easy reading. Interactive components and buttons need strategic placement to minimize errors and provide immediate feedback to users. The layout needs to function across various devices through support for multiple screen dimensions. The search function together with filter options should enable fast patient information retrieval. A uniform design structure prevents users from needing to learn system elements repeatedly. New users will find assistance through in-app tutorials and tooltips as well as a help center. Regular usability testing must include real users to identify problems which will enhance the system's performance and user experience.

ACCESSIBILITY

The system needs to support users with visual, physical, cognitive or auditory impairments and must comply with WCAG 2.1. The system should provide keyboard navigation functionality together with alt text and high contrast visuals and text-to-speech capabilities. The layout together with buttons should use straightforward language that is easy to understand. Users should have the ability to customize accessibility features including font size and contrast options. The application needs to provide multilingual support through translation tools which should not decrease performance levels. Help guides and tooltips must be localized. The application needs to operate on older devices and slower networks while maintaining its core features. Users with disabilities need to participate in testing to detect usability problems which will enhance actual system performance in areas with limited resources.

AVAILABILITY

The system should operate at 99.5% uptime with minimal downtime. The system should enable users to enter data and upload X-rays when offline before syncing the information when the device reconnects to the network. Hosting should be secure with backups. Load balancing is necessary to distribute traffic without performance degradation. The system needs failover mechanisms to maintain operation when failures occur. Maintenance tasks need to be planned to minimize system interruptions. Real-time monitoring is necessary to detect issues early. Regional data centers will enhance speed for users situated in different geographical areas. The implemented measures enhance system availability across different operational conditions particularly in field clinics with limited resources and during internet service disruptions.

Performance

The system must process chest X-ray images and deliver a preliminary pneumonia diagnosis within five seconds. It must support at least 50 concurrent users without performance degradation, ensuring health workers and remote clinicians experience consistent speeds. Caching techniques, image compression handling, and optimized database queries must be implemented to minimize loading times. Even with large image files, the system should remain responsive, preventing bottlenecks that could slow down critical patient assessments.

Security

The app must ensure the confidentiality, integrity, and availability of sensitive patient data. All data transmissions must use HTTPS encryption, and data stored on the server must be encrypted at rest. Role-based access control (RBAC) must limit health workers to accessing only their clinic's patient records, while clinicians have broader access for diagnosis reviews. Multi-factor authentication (MFA) must be implemented for remote clinician logins to prevent unauthorized access. Routine security audits and vulnerability assessments must be conducted to mitigate emerging threats and ensure the system remains secure.

Implementation part 4:

For the implementation stage, our Backend team developed a fully functional web application using the Flask framework and a SQLite database to manage patient records and X-ray data. The app features role-based dashboards for Doctors, Health Workers, and Patients, each with specific functionalities. A secure login system was implemented to restrict access to the relevant dashboards. Each user type logs in with predefined credentials stored in the database, and their session is managed using Flask's session handling. Based on the login, users are redirected to their role-specific dashboard.

Doctors can view a list of patients, check their details, access submitted chest X-rays, and enter diagnostic feedback. Health workers can register new patients, input details such as name, age, symptoms, and upload chest X-rays for analysis. Patients can log in to view their own medical summaries and diagnosis results.

The web app follows a modular structure with separate files for routing, forms, and templates. File uploads are validated and stored securely. The diagnosis result is generated using a trained AI model and shown instantly on the result page. This implementation phase focused on building a reliable and user-friendly solution tailored for real-world healthcare workflows in low-resource settings.

Login Credentials Used:

Doctor Login
Username: DoctorJohn
Password: 1234

Health Worker Login
Username: HealthworkerMike

Password: abcd

Patient Login
Username: Automatically created when a health worker registers a new patient
Password: 1234 (default for all patient

Machine learning model:

Our backend team developed and trained a Convolutional Neural Network (CNN) to classify chest X-ray images as either pneumonia-positive or normal. The model was trained on a labeled dataset and achieved strong validation accuracy. We evaluated its performance using confusion matrices and also compared it against models like VGG16, ResNet50, and Logistic Regression. Despite some models showing slightly higher accuracy, CNN was chosen for its balance between performance and efficiency in image-based medical tasks. The trained model was exported and successfully integrated into the web app to generate real-time diagnostic predictions from uploaded images.

User's stories implemented:

User Story 1: Radiologist Reviews Chest X-ray Reports
We built a feature where radiologists can log into the web app and upload chest X-ray images for analysis. When an image is uploaded, it's processed by our trained machine learning model, which detects any abnormalities and returns a diagnosis along with confidence scores. These results are stored in the SQLite database and linked to the radiologist's account. On the front end, the results are displayed clearly so the radiologist can review them. This helps speed up the diagnostic process and improves accuracy. In the video, we show this working by logging in as a radiologist and uploading a test image.

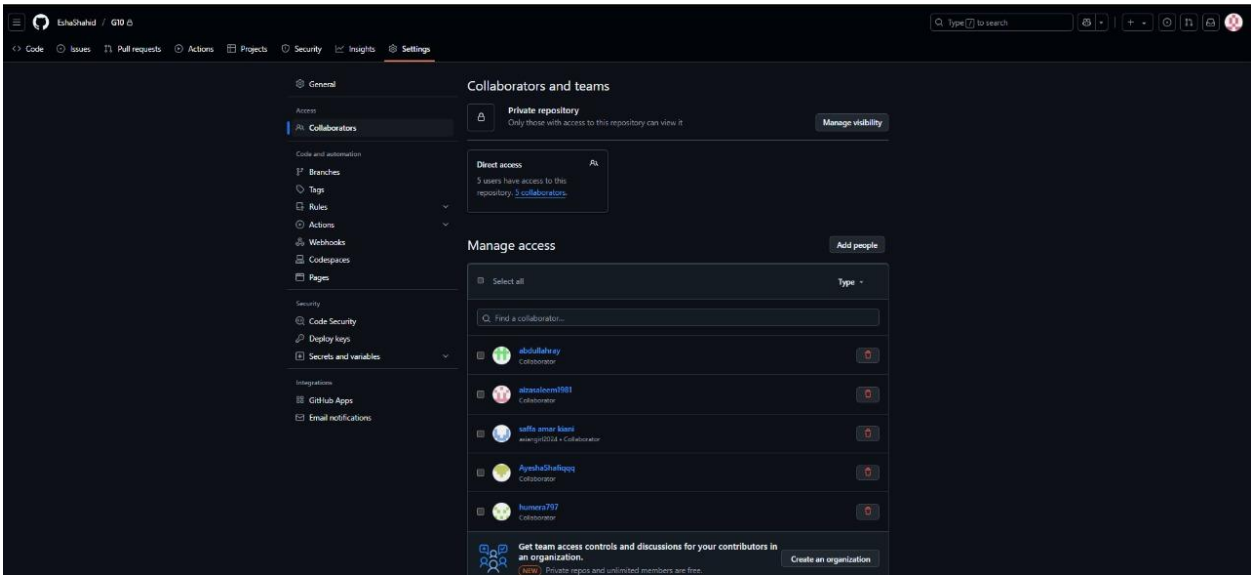
User Story 2: Patient Accesses Their Diagnosis Report
For patients, we created a login-based dashboard where they can securely access their personal diagnosis reports. Once signed in, they can view the AI-generated results of their X-ray scans. All data is retrieved from our SQLite database and is linked directly to the patient's ID, ensuring personalized and secure access. This feature helps patients stay informed about their health in a private and convenient way. In the group video, this functionality is demonstrated by logging in as a patient and navigating to the report section.

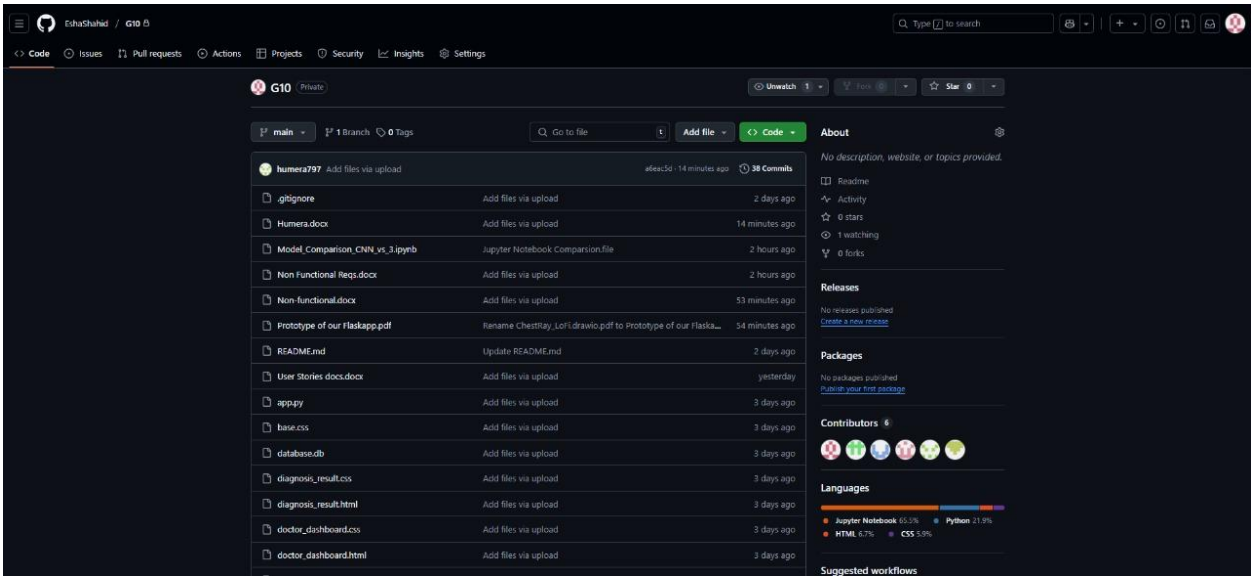
User Story 3: Doctor Reviews Patient's X-ray and Provides Recommendations
Doctors have a separate login area where they can view a list of patients under their care. After selecting a patient, they can review the AI-generated diagnosis from the uploaded X-rays. The system displays the results, and doctors are given a form where they can write treatment suggestions or additional notes. These notes are saved and stored securely in the database. This makes it easier for doctors to provide follow-up care. In our video, this part is shown by logging in as a doctor, opening a patient's record, and adding comments to their diagnosis.

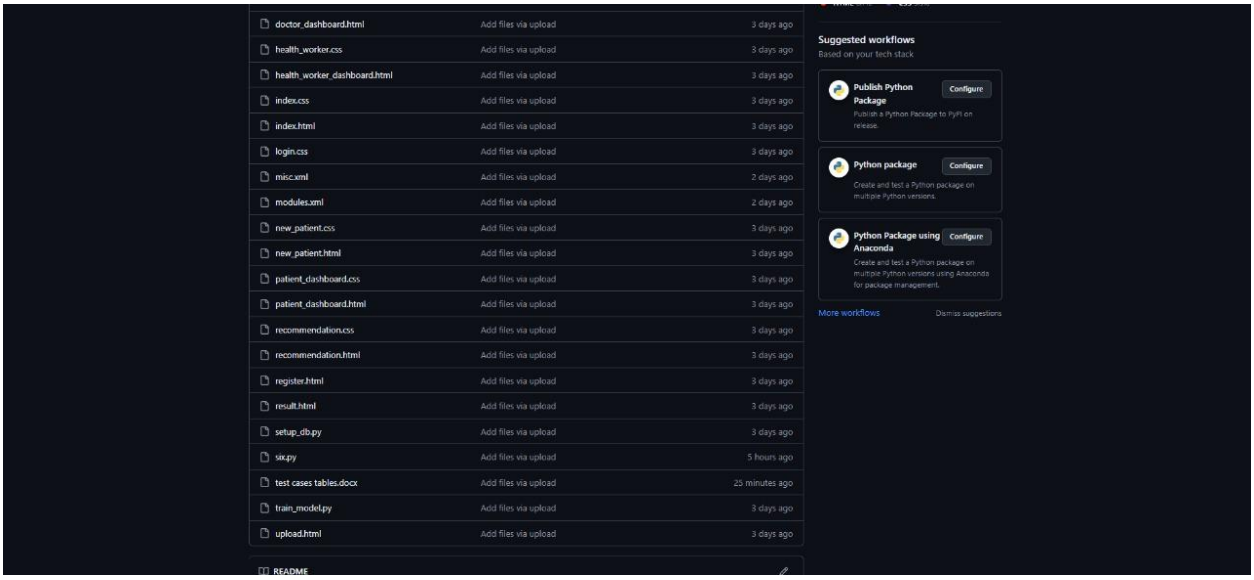
User Story 4: Developers Manage User Accounts and Permissions
In our project, Developer A and Developer B acted as administrators responsible for user management. Instead of a separate admin panel, we handled user roles and permissions directly through the backend

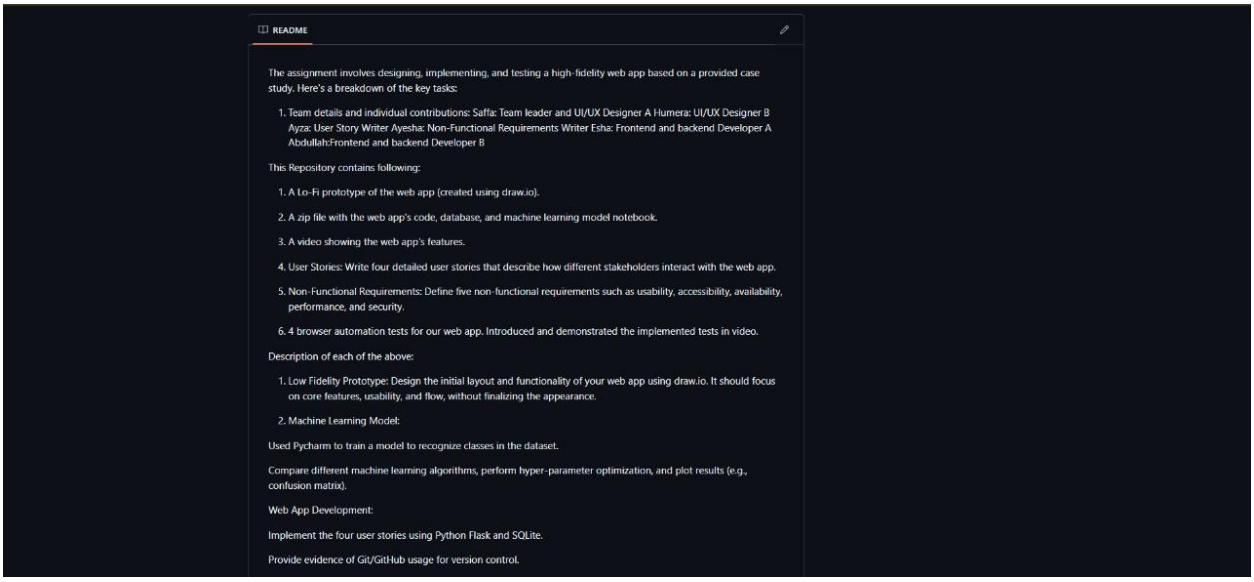
and database. We used the SQLite database to assign roles such as doctor, patient, or radiologist to each user during registration or via manual updates. This allowed us to control access levels without needing a dedicated admin interface. Developer A and B could view and modify user data by editing entries in the database. This approach kept the system simple while still ensuring that only authorized users could access certain features.

Evidence of Git/Github use:









README

The assignment involves designing, implementing, and testing a high-fidelity web app based on a provided case study. Here's a breakdown of the key tasks:

1. Team details and individual contributions: Saffa: Team leader and UI/UX Designer A Humera: UI/UX Designer B Ayza: User Story Writer Ayesha: Non-Functional Requirements Writer Esha: Frontend and backend Developer A Abdullah:Frontend and backend Developer B

This Repository contains following:

1. A Lo-Fi prototype of the web app (created using draw.io).
2. A zip file with the web app's code, database, and machine learning model notebook.
3. A video showing the web app's features.
4. User Stories: Write four detailed user stories that describe how different stakeholders interact with the web app.
5. Non-Functional Requirements: Define five non-functional requirements such as usability, accessibility, availability, performance, and security.
6. 4 browser automation tests for our web app. Introduced and demonstrated the implemented tests in video.

Description of each of the above:

1. Low Fidelity Prototype: Design the initial layout and functionality of your web app using draw.io. It should focus on core features, usability, and flow, without finalizing the appearance.
2. Machine Learning Model:

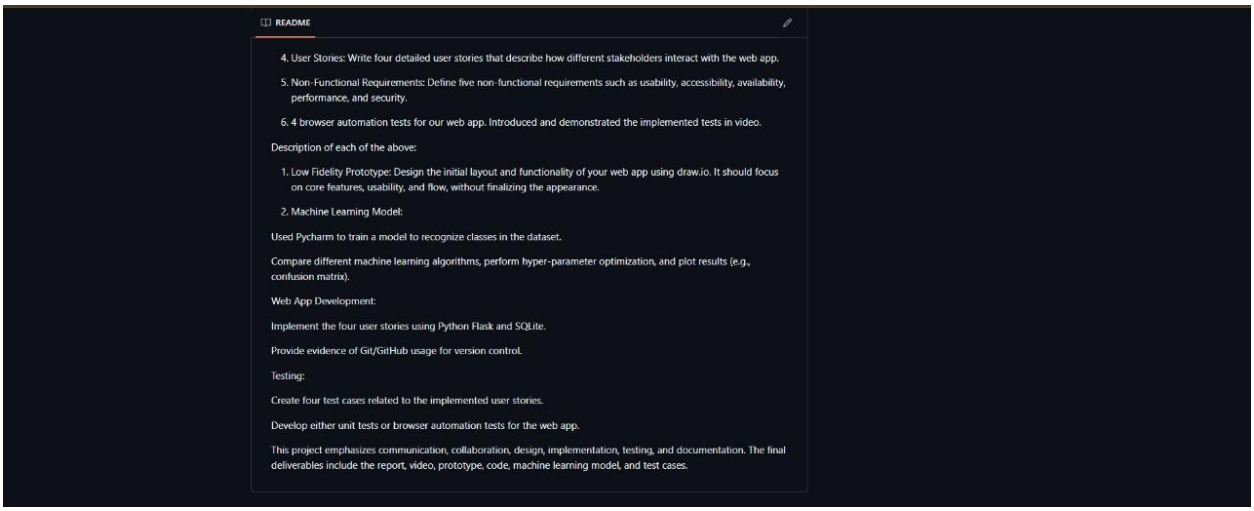
Used Pycharm to train a model to recognize classes in the dataset.

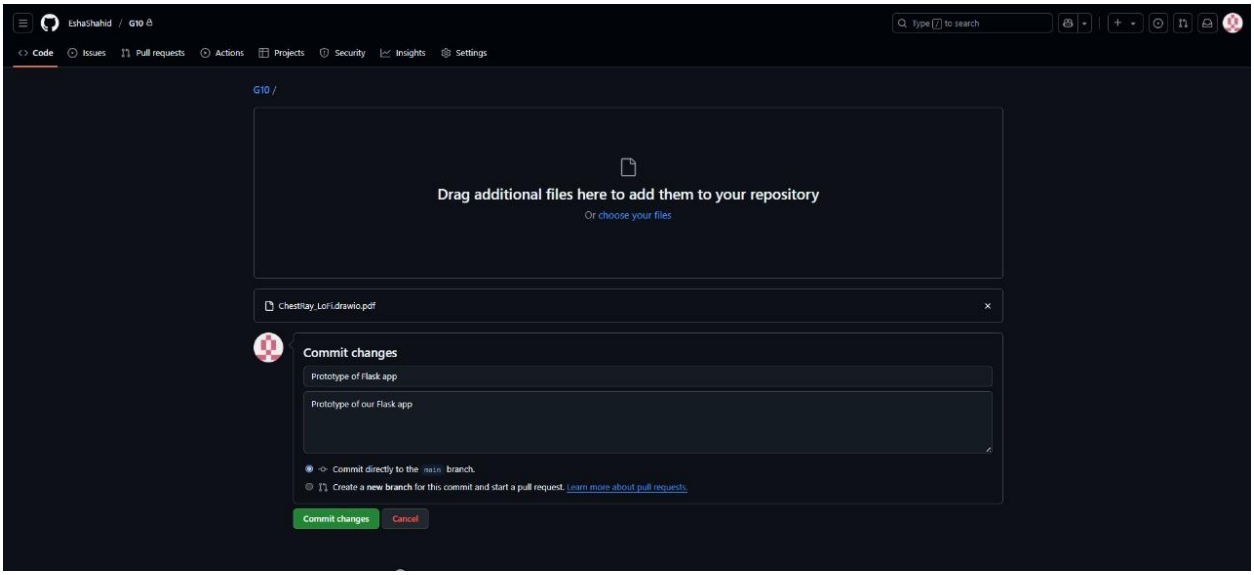
Compare different machine learning algorithms, perform hyper-parameter optimization, and plot results (e.g., confusion matrix).

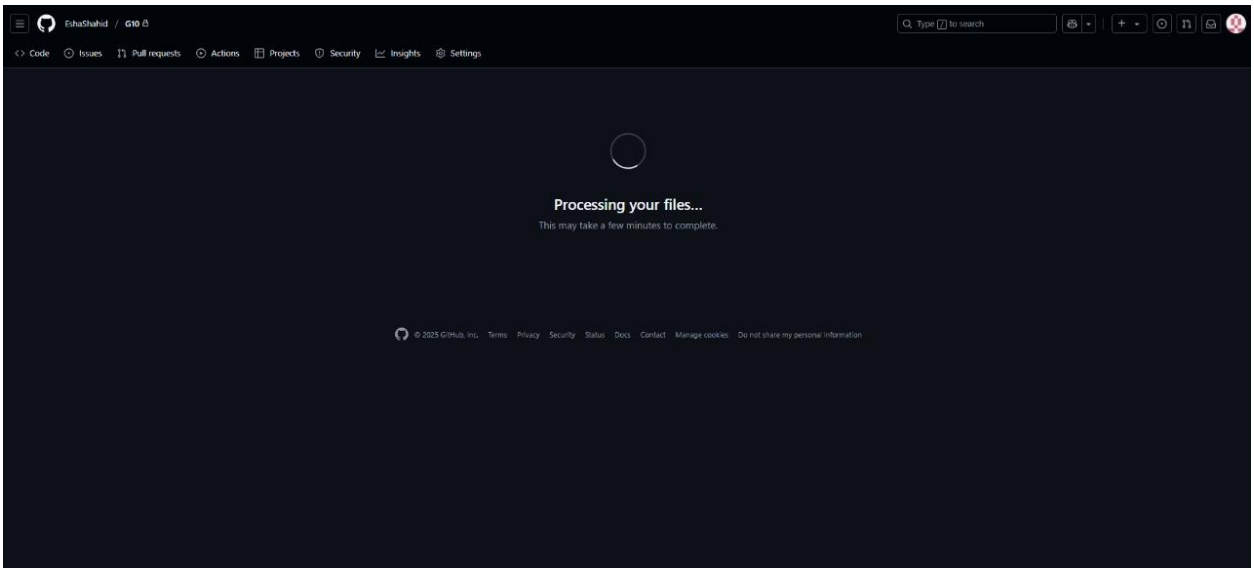
Web App Development:

Implement the four user stories using Python Flask and SQLite.

Provide evidence of Git/GitHub usage for version control.


README
4. User Stories: Write four detailed user stories that describe how different stakeholders interact with the web app.
5. Non-Functional Requirements: Define five non-functional requirements such as usability, accessibility, availability, performance, and security.
6. 4 browser automation tests for our web app. Introduced and demonstrated the implemented tests in video.
Description of each of the above:
1. Low Fidelity Prototype: Design the initial layout and functionality of your web app using draw.io. It should focus on core features, usability, and flow, without finalizing the appearance.
2. Machine Learning Model:
Used Pycharm to train a model to recognize classes in the dataset.
Compare different machine learning algorithms, perform hyper-parameter optimization, and plot results (e.g., confusion matrix).
Web App Development:
Implement the four user stories using Python Flask and SQLite.
Provide evidence of Git/GitHub usage for version control.
Testing:
Create four test cases related to the implemented user stories.
Develop either unit tests or browser automation tests for the web app.
This project emphasizes communication, collaboration, design, implementation, testing, and documentation. The final deliverables include the report, video, prototype, code, machine learning model, and test cases.


EshaShahid / G10
Type / to search
Code
Issues
Pull requests
Actions
Projects
Security
Insights
Settings
G10 /
Drag additional files here to add them to your repository
Or choose your files
ChestRay_LoFi.drawio.pdf
Commit changes
Prototype of Flask app
Prototype of our Flask app
Commit directly to the main branch.
Create a new branch for this commit and start a pull request. Learn more about pull requests.
Commit changes
Cancel



Throughout our project, Git and GitHub were central to our team's workflow, ensuring transparency, coordination, and version control. We maintained a private GitHub repository (EshaShahid/G10) where all six team members were granted collaborator access. GitHub was not only used for code uploads but also as a full project management platform. We created and assigned issues to track progress, clarify responsibilities, and document challenges and solutions. This ensured tasks were systematically managed and visible to all.

Commit history demonstrates consistent, meaningful contributions from all members, with descriptive messages documenting the purpose of each change—such as implementing Flask routes, uploading the trained CNN model, or refining front-end components. Each update reflected deliberate planning, and comments within commits offered insights into our thought process and improvements made based on peer feedback or test results.

We used branches to safely develop features in parallel, avoiding conflicts and enabling thorough reviews before merging. The README file includes individual roles, team responsibilities, and project structure, highlighting our organized approach.

By embedding GitHub into every stage—from planning to deployment—we demonstrated a professional-level use of development tools. This enabled collaborative reflection, streamlined workflows, and ultimately ensured a cohesive, high-quality final product.

Testing (test cases):

Test Case Id.	Test Scenario and Objective	Precondition (if any)	Test Steps	Input Data (if any)	Expected Result

PN001	Login with valid credentials to ensure that a registered user can successfully log in.	User must be registered with valid credentials.	● Navigate to the login page. ● Enter a registered email and password. ● Click the "Login" button.	● Username: Doctorjohn ● Password: 1234	User successfully logs in and is redirected to the homepage.
PN002	Doctor recommendation – To ensure that the doctor is able to view analysis and submit recommendations.	Doctor must be logged in and reports must be available to review.	● Login as doctor. ● Open patient dashboard. ● Select a patient. ● Add recommendation. ● Save changes and submit.	Medical recommendation.	Recommendations are saved and the patient's record is updated.
PN003	Health worker adds a new patient record – To ensure that a health worker can successfully add a new patient's information into the system.	Health worker must be logged in with the appropriate role and permissions.	● Login as health worker. ● Navigate to the "Add New Patient" section. ● Enter patient details including name, age, gender, contact information, and medical history. ● Click "Submit" or "Save" to add the record	● Name: John Doe ● Age: 45 ● Gender: Male ● Contact: johndoe@email.com ● Medical History: Hypertension, Diabetes	Patient record is successfully added to the system and becomes visible in the patient list.
PN004	Patient views report – Patient should be able to access their report if it exists.	Patient must have an account and a full diagnosis report uploaded.	● Go to login page. ● Login as patient using correct credentials.	● Username: Username ● Password: Password	Report is displayed with patient details, diagnosis, symptoms, status, and doctor's recommendations.

Automated Test Code:

Table listing:

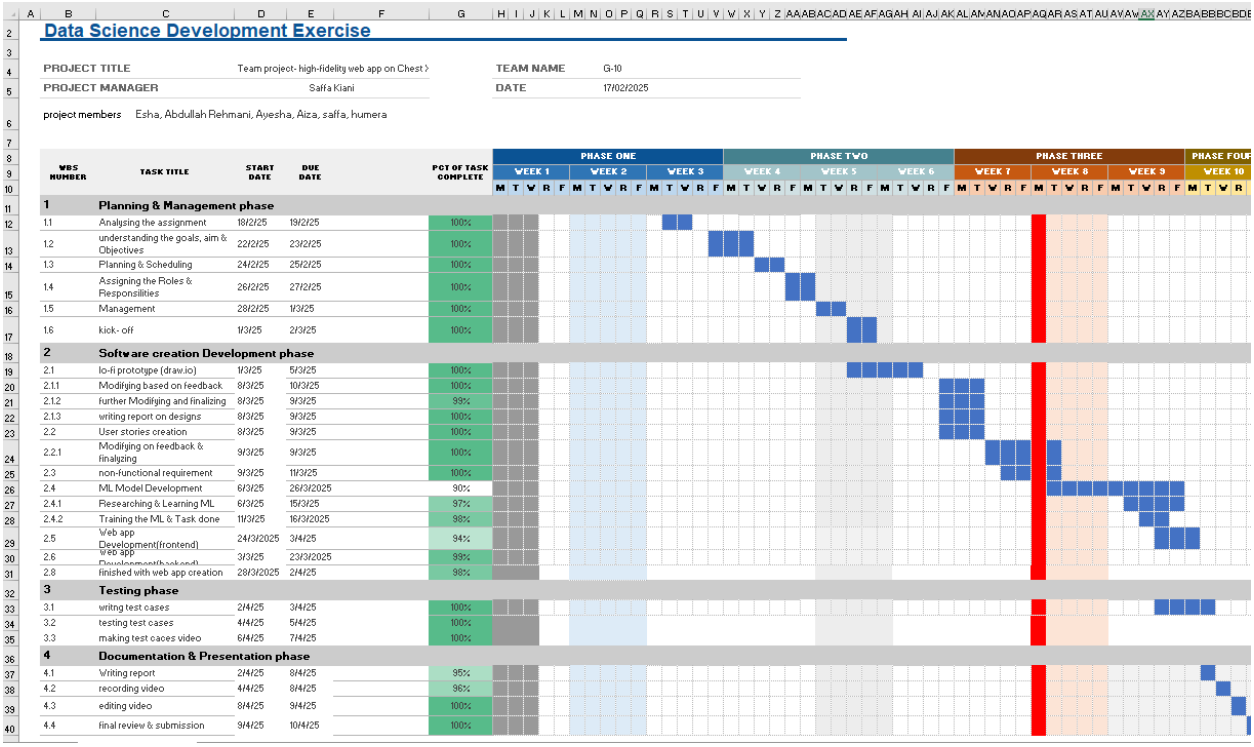
test id	text description	timestamps in video
to1	login credentials	22:12—23:25
to2	doctor's recommendation	23:29---25:12
to3	patient dashboard	25:18---26----11
to4	health worker adds new patient	26: 41---28:04

Conclusion:

This project successfully demonstrated the creation of a Flask web application powered by machine learning and focused to facilitate diagnostic processes involving chest X-ray imagery. We were able to create a working system that meets the needs of a variety of stakeholders, including patients, doctors, radiologists, and administrators, by merging artificial intelligence and user-centered design.

Throughout the development process, we used agile approaches to define and implement user stories, design a clear and accessible interface, and build a backend that securely processes data using SQLite. Our machine learning pipeline consisted of assessing multiple methods, optimizing performance, and finally integrating the best model into the web application for real-time inference.

Gantt chart



Data Science Development Exercise

PROJECT TITLE	Team project- high-fidelity web app on Chest X	TEAM NAME	G-10
PROJECT MANAGER	Saffa Kiani	DATE	17/02/2025

project members: Esha, Abdullah Rehmani, Ayesha, Aiza, saffa, humera

WBS NUMBER	TASK TITLE	START DATE	DUE DATE	PCT OF TASK COMPLETE
1	**Planning & Management phase**			
1.1	Analysing the assignment	18/2/25	19/2/25	100%
1.2	understanding the goals, aim & Objectives	22/2/25	23/2/25	100%
1.3	Planning & Scheduling	24/2/25	25/2/25	100%
1.4	Assigning the Roles & Responsilities	26/2/25	27/2/25	100%
1.5	Management	28/2/25	1/3/25	100%
1.6	kick- off	1/3/25	2/3/25	100%
2	**Software creation Development phase**			
2.1	lo-fi prototype (draw.io)	1/3/25	5/3/25	100%
2.1.1	Modifying based on feedback	8/3/25	10/3/25	100%
2.1.2	further Modifying and finalizing	8/3/25	9/3/25	99%
2.1.3	writing report on designs	8/3/25	9/3/25	100%
2.2	User stories creation	8/3/25	9/3/25	100%
2.2.1	Modifying on feedback & finalizing	9/3/25	9/3/25	100%
2.3	non-functional requirement	9/3/25	11/3/25	100%
2.4	ML Model Development	6/3/25	26/3/2025	90%
2.4.1	Researching & Learning ML	6/3/25	15/3/25	97%
2.4.2	Training the ML & Task done	11/3/25	16/3/2025	98%
2.5	Web app Development(frontend)	24/3/2025	3/4/25	94%
2.6	Web app Development(backend)	3/3/25	23/3/2025	99%
2.8	finished with web app creation	28/3/2025	2/4/25	98%
3	**Testing phase**			
3.1	writng test cases	2/4/25	3/4/25	100%
3.2	testing test cases	4/4/25	5/4/25	100%
3.3	making test caces video	6/4/25	7/4/25	100%
4	**Documentation & Presentation phase**			
4.1	Writing report	2/4/25	8/4/25	95%
4.2	recording video	4/4/25	8/4/25	96%
4.3	editing video	8/4/25	9/4/25	100%
4.4	final review & submission	9/4/25	10/4/25	100%

PHASE ONE: WEEK 1, WEEK 2, WEEK 3 | PHASE TWO: WEEK 4, WEEK 5, WEEK 6 | PHASE THREE: WEEK 7, WEEK 8, WEEK 9 | PHASE FOUR: WEEK 10